

Nonstandard GPS Receiver Using ESP32

Somnath Bera

Sometimes things do not fall in place and we curse ourselves for taking the wrong turn or making the wrong choice. But in P.B. Shelly's words, when the realisation later dawns on us we feel like as described below by the poet.

*We look before and after
And pine for what is not
Our sincerest laughter with some
pain is fraught*

*Our sweetest songs are those that
tell about our saddest thoughts*

Once I bought a pair of Arduino-compatible GPS receivers from *aliexpress.com* at a throwaway price and then the whole problem started. The GPS receivers do not talk to any standard program. So, none of the Arduino programs such as Adafruit-GPS, NeoGPS, TinyGPS, TinyGPS ++, or the oldest GPS could talk to the module to decipher data. Frustrated, I asked for the refund of money—stat-

ing the reason that the receivers were non-compatible with Arduino. (That's what they were told, actually!)

The seller immediately made the refund, which was a small amount, and I simply set the receivers aside to finally throw them in the wastebasket someday. But what happened a couple of months later is worth sharing!

I was experimenting with a project where precise time-keeping was necessary, and I lay my eyes on these small 'good-for-nothing' GPS receivers. I picked one of them and, using a USB-to-serial FTDI board, I connected the module to the raw serial terminal of my computer (3.3V + Gnd + Tx + Rx pins used). I set the terminal speed as 9600-8N1 and I found all those NMEA (National Marine Electronics Association) data (see Fig. 1) arriving at the GTK terminal of my Ubuntu PC. Two different types of GPS modules (A and B) used by me are shown in Fig. 3. You can use anyone of the modules for this project.

Though the data was not at all clear to me (what was what) at first, a close look later showed that the figures highlighted in yellow in Fig. 1 were UPC (United Pacific Time) and date strings.

For a better picture, I connected the GPS module to Serial2 hardware port of ESP32, and the data now arrived very clearly on the Arduino terminal. On looking carefully, all that GPS data started looking meaningful:

A (active or fixed), longitude, date, number of satellite-in-view, speed, mean-sea-level, etc. All these data were arriving at fixed position with fix frequencies as shown in Fig. 2.

Different type of NMEA data started arriving on Serial2 port of the ESP32 board that was used during testing. The data was different from the normal NEMA data, and that was the reason why normal GPS programs could not decipher them at all.

After a few cursory studies of the data that was arriving, I found that important parameters like latitude, longitude, hhmmss (hours, minutes, seconds), ddmmyy (day, month, year), altitude, number of satellites in view, fix, km/hour speed could be located easily in the data.

Once I knew where these data were located in the once 'junk-like' serial NMEA data stream, I made an attempt to pluck out the now those useful data for the project.

In an Arduino, for connecting any normal UART module, it is always good to avoid the native Rx and Tx (D0 and D1) pins as these two pins are used while uploading sketch (software) from a computer. One has to make those pins free while uploading the sketch, else, it will not upload. Therefore, it is convenient to connect any UART device to some other GPIOs, and for that people mostly use programs like softwareserial or neoserial. Unfortunately, these software do not work on ESP32.

ESP32 has three hardware serials, and any pin can be set for the task with the following commands:

```
Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2); // 17,12,4,35 pins are OK
```

You can set any pin for the Tx and Rx ports. One more interesting fact about the NMEA is that, the data that arrives endlessly is in CSV form. Therefore, separating it on CSV is easy! For

```
$GNGGA,172549.00,,,,,0.00,99.99,,,,,*74
$GNGSA,A,1,,,,,,,,,99.99,99.99,99.99*2E
$GNGSA,A,1,,,,,,,,,99.99,99.99,99.99*2E
$GPGSV,1,1,01,31,,,26*7E
$GLGSV,1,1,01,,,26*60
$GNGLL,,,,,172549.00,V,N*58
$GNRMC,172550.00,V,,,,,030719,,,N*6B
$GNVTG,,,,,N*2E
$GNGGA,172550.00,,,,,0.00,99.99,,,,,*7C
$GNGSA,A,1,,,,,,,,,99.99,99.99,99.99*2E
$GNGSA,A,1,,,,,,,,,99.99,99.99,99.99*2E
$GPGSV,1,1,01,31,,,26*7E
$GLGSV,1,1,01,,,27*61
$GNGLL,,,,,172550.00,V,N*50
```

Fig. 1: Sample data from GPS receiver

```
$GNRMC,173206.00,A,2405.32467,N,08238.89017,E,0.177,030719,,,D*66
$GNVTG,,T,M,0.177,N,0.328,K,D*30
$GNGGA,173206.00,2405.32467,N,08238.89017,E,2,12.0.69,281.5,M,-53.1,M,0.000*69
$GNGSA,A,3,31,10,14,32,20,18,25,01,11,26,22,40,1.30,0.69,1.11*1A
$GNGSA,1,14,295,25,12,06,051,*74
$GPGSV,4,2,16,14,53,338,26,18,36,296,29,20,31,120,32,21,04,161,23*7F
$GPGSV,4,3,16,22,09,315,21,25,24,085,27,26,12,175,25,27,02,221,18*71
$GPGSV,4,4,16,31,74,205,26,32,49,013,25,40,48,232,28,41,62,179,*7A
$GLGSV,3,1,09,69,13,088,,70,42,031,,71,26,329,,73,50,161,29*66
$GLGSV,3,2,09,74,11,188,,79,02,026,,80,47,053,,85,21,230,17*63
$GLGSV,3,3,09,86,22,290,*59
$GNGLL,2405.32467,N,08238.89017,E,173206.00,A,D*72
```

Fig. 2: Detailed output from GPS receiver